

zadania_lab_12

Zadanie 1 — obliczanie wartości wielomianu schematem Hornera

Polecenie:

Napisz program obliczający wartość wielomianu w punkcie z wykorzystaniem schematu Hornera.

Na czym polega schemat Hornera?

Schemat Hornera to metoda szybkiego obliczania wartości wielomianu w danym punkcie.

Dany jest wielomian:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

Zamiast liczyć osobno każdą potęgę z , przekształcamy wielomian do postaci:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Dzięki temu w każdym kroku wykonujemy tylko jedno mnożenie i jedno dodawanie.

Algorytm ze slajdu

Schemat Hornera działa według zasady:

$$p \leftarrow a_n$$

Następnie dla kolejnych współczynników:

$$p \leftarrow p \cdot z + a_i$$

Na końcu otrzymujemy:

$$p = P_n(z)$$

czyli wartość wielomianu w punkcie z .

Ważne

Współczynniki wielomianu zapisujemy w liście **od najwyższej potęgi do wyrazu wolnego**.

Na przykład dla wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

lista współczynników wygląda tak:

```
wspolczynniki = [3, 2, -1, 5]
```

czyli:

- 3 — współczynnik przy z^3 ,
- 2 — współczynnik przy z^2 ,
- -1 — współczynnik przy z ,
- 5 — wyraz wolny.

Przykład ze slajdu

Dany jest wielomian:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

Chcemy obliczyć jego wartość dla:

$$z = 2$$

Schemat Hornera:

$$p = 3$$

$$p = 3 \cdot 2 + 2 = 8$$

$$p = 8 \cdot 2 - 1 = 15$$

$$p = 15 \cdot 2 + 5 = 35$$

Zatem:

$$P(2) = 35$$

Kod programu

```
def schemat_Hornera(wspolczynniki, z): # definiujemy funkcję obliczającą wartość
    wielomianu w punkcie z
    p = wspolczynniki[0] # jako początkową wartość p przyjmujemy pierwszy
    współczynnik, czyli ten przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych
    współczynnikach od drugiego elementu listy
        p = p * z + wspolczynniki[i] # wykonujemy krok schematu Hornera: p = p*z
        + kolejny współczynnik

    return p # zwracamy końcową wartość, czyli P(z)
```

Test programu

PYTHON

```
wspolczynniki = [3, 2, -1, 5] # współczynniki wielomianu P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt, w którym liczymy wartość wielomianu; tutaj z = 2

wynik = schemat_Hornera(wspolczynniki, z) # wywołujemy funkcję schematu Hornera

print("P(z) =", wynik) # wypisujemy wynik
```

Wynik:

$P(z) = (35+0j)$

Zapis:

$35 + 0j$

oznacza po prostu liczbę:

35

Python pokazuje `+0j`, ponieważ użyto typu `complex`.

Cały program

PYTHON

```
# zad 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja oblicza wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potęgde

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok Hornera

    return p # zwracamy wartość wielomianu

wspolczynniki = [3, 2, -1, 5] # wielomian P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt z = 2

wynik = schemat_Hornera(wspolczynniki, z) # obliczamy wartość wielomianu

print("P(z) =", wynik) # wypisujemy wynik
```

Wnioski

Schemat Hornera pozwala szybko obliczyć wartość wielomianu w punkcie.

Zamiast liczyć potęgi osobno, np.:

$$z^3, z^2, z$$

wykonujemy kolejne działania postaci:

$$p \leftarrow pz + a_i$$

Dzięki temu program jest krótszy, szybszy i wygodny także dla liczb zespolonych.

Dla przykładowego wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

oraz:

$$z = 2$$

otrzymujemy:

$$P(2) = 35$$

Poniżej masz gotową notatkę do **zadania 2**, w takim samym stylu jak do zadania 1 i zgodnie ze slajdami o schemacie Hornera.

Zadanie 2 — obliczanie wartości pierwszej i drugiej pochodnej wielomianu w punkcie

Polecenie:

Napisz program obliczający wartość pierwszej i drugiej pochodnej wielomianu w punkcie.

O co chodzi w zadaniu?

W zadaniu 1 schemat Hornera służył do obliczania wartości wielomianu:

$$P(z)$$

W zadaniu 2 rozszerzamy schemat Hornera tak, aby obliczyć jednocześnie:

$$P(z)$$

$$P'(z)$$

$$P''(z)$$

czyli:

- wartość wielomianu w punkcie,
- wartość pierwszej pochodnej w punkcie,
- wartość drugiej pochodnej w punkcie.

Przypomnienie schematu Hornera

Dany jest wielomian:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

W schemacie Hornera zapisujemy go w postaci:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Podstawowy krok schematu Hornera ma postać:

$$p_{\text{nowe}} = p_{\text{stare}} \cdot z + a_i$$

Ten wzór służy do obliczania wartości wielomianu.

Skąd biorą się wzory na pochodne?

W każdym kroku Hornera mamy:

$$p_{\text{nowe}}(z) = p_{\text{stare}}(z) \cdot z + a_i$$

Teraz różniczkujemy ten wzór.

Pierwsza pochodna

Różniczkujemy:

$$p_{\text{nowe}}(z) = p_{\text{stare}}(z) \cdot z + a_i$$

Pochodna stałej (a_i) wynosi 0.

Z pochodnej iloczynu:

$$(uv)' = u'v + uv'$$

otrzymujemy:

$$p' * \text{nowe}(z) = p' * \text{stare}(z) \cdot z + p_{\text{stare}}(z)$$

W programie zapisujemy to jako:

```
dp = dp * z + p
```

PYTHON

gdzie:

- **dp** oznacza pierwszą pochodną,
- **p** oznacza wartość wielomianu z poprzedniego kroku.

Druga pochodna

Teraz różniczkujemy wzór na pierwszą pochodną:

$$p' * \text{nowe}(z) = p' * \text{stare}(z) \cdot z + p_{\text{stare}}(z)$$

Otrzymujemy:

$$p'' * \text{nowe}(z) = p'' * \text{stare}(z) \cdot z + p' * \text{stare}(z) + p' * \text{stare}(z)$$

czyli:

$$p'' * \text{nowe}(z) = p'' * \text{stare}(z) \cdot z + 2p'_{\text{stare}}(z)$$

W programie zapisujemy to jako:

```
ddp = ddp * z + 2 * dp
```

PYTHON

gdzie:

- **ddp** oznacza drugą pochodną,
- **dp** oznacza pierwszą pochodną z poprzedniego kroku.

Ważna kolejność działań w programie

W pętli najpierw aktualizujemy drugą pochodną, potem pierwszą pochodną, a dopiero na końcu wartość wielomianu:

```
ddp = ddp * z + 2 * dp
dp = dp * z + p
p = p * z + wspolczynniki[i]
```

PYTHON

Taka kolejność jest ważna, ponieważ **ddp** musi korzystać ze starej wartości **dp**, a **dp** musi korzystać ze starej wartości **p**.

Kod programu

```

PYTHON
def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja oblicza P(z), P'(z)
    oraz P''(z)
    p = wspolczynniki[0] # p przechowuje aktualną wartość wielomianu
    dp = 0 # dp przechowuje aktualną wartość pierwszej pochodnej
    ddp = 0 # ddp przechowuje aktualną wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych
        współczynnikach
            ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
            dp = dp * z + p # aktualizujemy pierwszą pochodną
            p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu schematem
        Hornera

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)

```

Przykład

Weźmy wielomian ze slajdu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

Współczynniki zapisujemy od najwyższej potęgi do wyrazu wolnego:

```

wspolczynniki = [3, 2, -1, 5]

```

PYTHON

Obliczamy wartości dla:

$$z = 2$$

Obliczenia analityczne do sprawdzenia

Najpierw liczymy wartość wielomianu:

$$P(2) = 3 \cdot 2^3 + 2 \cdot 2^2 - 2 + 5$$

$$P(2) = 24 + 8 - 2 + 5 = 35$$

Pierwsza pochodna:

$$P'(z) = 9z^2 + 4z - 1$$

$$P'(2) = 9 \cdot 2^2 + 4 \cdot 2 - 1$$

$$P'(2) = 36 + 8 - 1 = 43$$

Druga pochodna:

$$P''(z) = 18z + 4$$

$$P''(2) = 18 \cdot 2 + 4$$

$$P''(2) = 36 + 4 = 40$$

Czyli oczekujemy:

$$P(2) = 35$$

$$P'(2) = 43$$

$$P''(2) = 40$$

Test programu

```
PYTHON
wspolczynniki = [3, 2, -1, 5] # współczynniki wielomianu P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt, w którym liczymy wartości

wartosc, pierwsza_pochodna, druga_pochodna =
schemat_Hornera_pochodne(wspolczynniki, z) # obliczamy P(z), P'(z), P''(z)

print("P(z) =", wartosc) # wypisujemy wartość wielomianu
print("P'(z) =", pierwsza_pochodna) # wypisujemy wartość pierwszej pochodnej
print("P''(z) =", druga_pochodna) # wypisujemy wartość drugiej pochodnej
```

Wynik:

```
P(z) = (35+0j)
P'(z) = (43+0j)
P''(z) = (40+0j)
```

Zapis `+0j` oznacza, że wynik jest liczbą zespoloną z częścią urojoną równą 0.

Cały program


```
# zad 2 -----
```

```
def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja oblicza P(z), P'(z),
P''(z)
    p = wspolczynniki[0] # początkowa wartość wielomianu to współczynnik przy
najwyższej potędze
    dp = 0 # początkowa wartość pierwszej pochodnej
    ddp = 0 # początkowa wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy przez kolejne
współczynniki
        ddp = ddp * z + 2 * dp # obliczamy nową wartość drugiej pochodnej
        dp = dp * z + p # obliczamy nową wartość pierwszej pochodnej
        p = p * z + wspolczynniki[i] # obliczamy nową wartość wielomianu

    return p, dp, ddp # zwracamy wartość wielomianu, pierwszej pochodnej i drugiej
pochodnej

wspolczynniki = [3, 2, -1, 5] # wielomian P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt z=2

wartosc, pierwsza_pochodna, druga_pochodna =
schemat_Hornera_pochodne(wspolczynniki, z) # wywołujemy funkcję

print("P(z) =", wartosc) # wypisujemy P(z)
print("P'(z) =", pierwsza_pochodna) # wypisujemy P'(z)
print("P''(z) =", druga_pochodna) # wypisujemy P''(z)
```

Wnioski

W zadaniu 2 rozszerzono schemat Hornera tak, aby oprócz wartości wielomianu obliczał także pierwszą i drugą pochodną.

Dzięki temu jednym przejściem przez listę współczynników można obliczyć:

$$P(z), \quad P'(z), \quad P''(z)$$

Dla przykładowego wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

oraz punktu:

$$z = 2$$

otrzymujemy:

$$P(2) = 35$$

$$P'(2) = 43$$

$$P''(2) = 40$$

Kod działa również dla liczb zespolonych, ponieważ punkt (z) można zapisać jako:

```
z = complex(2, 0)
```

PYTHON

Zadanie 3 — metoda Laguerre'a

Polecenie:

Zaimplementuj metodę Laguerre'a służącą do znajdowania pierwiastków wielomianów, również zespolonych.

O co chodzi w zadaniu?

Wykorzystujemy funkcje z zadania 1 i 2, bo metoda Laguerre'a potrzebuje wartości:

$$P(z), \quad P'(z), \quad P''(z)$$

czyli dokładnie tego, co liczy Horner rozszerzony z zadania 2.

Mamy wielomian, np.:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

Wiemy, że jego pierwiastki to:

$$z_1 = -1, \quad z_2 = -2, \quad z_3 = -1 + 2i, \quad z_4 = -1 - 2i$$

Ale w praktyce program nie zna rozkładu wielomianu, tylko jego współczynniki:

$$1, 5, 13, 19, 10$$

czyli:

$$P(z) = 1z^4 + 5z^3 + 13z^2 + 19z + 10$$

Zadaniem programu jest znalezienie **jednego pierwiastka** metodą Laguerre'a.

Wzory

Dla wielomianu stopnia n w punkcie z liczymy:

$$G = \frac{P'(z)}{P(z)}$$

oraz:

$$H = G^2 - \frac{P''(z)}{P(z)}$$

Następnie liczymy poprawkę:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH-G^2)}}$$

Znak w mianowniku wybieramy tak, aby mianownik miał **większy moduł**:

$$\left| G \pm \sqrt{(n-1)(nH-G^2)} \right|$$

Potem obliczamy nowe przybliżenie:

$$z_{\text{new}} = z_{\text{old}} - a$$

Iteracje kończymy, gdy:

$$|a| < \varepsilon$$

```
# zad 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja licząca wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok schematu Hornera

    return p # zwracamy wartość wielomianu P(z)

# zad 2 -----

def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja licząca P(z), P'(z), P''(z)
    p = wspolczynniki[0] # p oznacza aktualną wartość wielomianu
    dp = 0 # dp oznacza aktualną wartość pierwszej pochodnej
    ddp = 0 # ddp oznacza aktualną wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
        dp = dp * z + p # aktualizujemy pierwszą pochodną
        p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)

# zad 3 -----

import cmath # importujemy cmath, bo metoda Laguerre'a może używać pierwiastków zespolonych

def metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0, epsilon=1e-6, max_iteracji=100):
    z = complex(z0) # zamieniamy punkt startowy na liczbę zespoloną
    n = len(wspolczynniki) - 1 # stopień wielomianu to liczba współczynników minus 1

    for k in range(max_iteracji): # wykonujemy kolejne iteracje metody Laguerre'a
        P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z) # liczymy P(z), P'(z), P''(z)

        if abs(P) < epsilon: # jeśli wartość wielomianu jest bardzo bliska 0
            return z # to uznajemy, że z jest pierwiastkiem

    G = P_prim / P # liczymy G = P'(z) / P(z)
    H = G**2 - P_2prim / P # liczymy H = G^2 - P''(z) / P(z)
```

```

    pierwiastek = cmath.sqrt((n - 1) * (n * H - G**2)) # liczymy wyrażenie
    pod pierwiastkiem ze wzoru

    mianownik_plus = G + pierwiastek # pierwszy możliwy mianownik
    mianownik_minus = G - pierwiastek # drugi możliwy mianownik

    if abs(mianownik_plus) > abs(mianownik_minus): # wybieramy mianownik o
    większym module
        mianownik = mianownik_plus # jeśli większy jest plus, wybieramy plus
    else:
        mianownik = mianownik_minus # w przeciwnym razie wybieramy minus

    if abs(mianownik) == 0: # zabezpieczenie przed dzieleniem przez zero
        z = z + complex(epsilon, epsilon) # lekko przesuwamy punkt
        continue # przechodzimy do następnej iteracji

    a = n / mianownik # liczymy poprawkę a ze wzoru Laguerre'a

    z_nowe = z - a # liczymy nowe przybliżenie pierwiastka

    if abs(a) < epsilon: # jeżeli poprawka jest bardzo mała
        return z_nowe # kończymy, bo znaleźliśmy pierwiastek

    z = z_nowe # aktualizujemy z i przechodzimy do kolejnej iteracji

    return z # jeśli nie spełniono warunku wcześniej, zwracamy ostatnie
    przybliżenie

# DODATKOWY TEST
print("----- ZADANIE 3 -----")

# Przykład ze slajdu:
#  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ 

wspolczynniki = [1, 5, 13, 19, 10] # współczynniki wielomianu od najwyższej
potęgi do wyrazu wolnego

z0 = complex(0, 0) # punkt startowy  $z_0 = 0$ 

pierwiastek = metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0) # szukamy
jednego pierwiastka

print("Wielomian:  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ ") # wypisujemy wielomian
print("Punkt startowy  $z_0 =$ ", z0) # wypisujemy punkt startowy
print("Znaleziony pierwiastek =", pierwiastek) # wypisujemy znaleziony pierwiastek
print("Sprawdzenie  $P(\text{pierwiastek}) =$ ", schemat_Hornera(wspolczynniki, pierwiastek))
# sprawdzamy, czy  $P(z)$  jest bliskie 0

```

Co powinno wyjść?

Dla wielomianu ze slajdu:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

metoda startując od:

$$z_0 = 0$$

powinna znaleźć pierwiastek bliski:

$$z = -1$$

Czyli wynik może wyglądać np. tak:

```
Znaleziony pierwiastek = (-1.0000000000000002+0j)
Sprawdzenie P(pierwiastek) = bardzo mała liczba bliska 0
```

Jeżeli pojawi się zapis:

$$(-1+0j)$$

to oznacza:

$$-1 + 0i$$

czyli po prostu:

$$-1$$

Łopatologiczne wyjaśnienie działania

Metoda Laguerre'a działa iteracyjnie, czyli nie znajduje pierwiastka od razu, tylko poprawia kolejne przybliżenia.

Zaczynamy od jakiegoś punktu, np.:

$$z_0 = 0$$

Potem program liczy w tym punkcie:

$$P(z), \quad P'(z), \quad P''(z)$$

Na podstawie tych wartości wylicza poprawkę (a). Ta poprawka mówi, o ile trzeba przesunąć aktualne przybliżenie.

Nowe przybliżenie liczymy tak:

$$z_{\text{new}} = z_{\text{old}} - a$$

Jeśli poprawka (a) jest jeszcze duża, program liczy dalej.

Jeśli poprawka jest bardzo mała, czyli:

$$|a| < \varepsilon$$

to znaczy, że jesteśmy już bardzo blisko pierwiastka.

Ważne

To zadanie znajduje **jeden pierwiastek**.

Dopiero w zadaniu 4 robi się deflację, czyli obniżanie stopnia wielomianu po znalezieniu pierwiastka, żeby znaleźć wszystkie pierwiastki.

Poniżej masz **zadanie 4 zrobione najprościej jak się da**, ale zgodnie ze slajdami: metoda Laguerre'a znajduje jeden pierwiastek, potem robimy **deflację**, czyli obniżamy stopień wielomianu, i powtarzamy aż zostanie wielomian stopnia 2. Dla stopnia 2 używamy zwykłego wzoru z deltą.

Zadanie 4 — wszystkie pierwiastki metodą Laguerre'a

Polecenie:

Zmodyfikuj program z poprzedniego zadania, aby wyznaczał wszystkie pierwiastki wielomianu (również zespolone).

O co chodzi w zadaniu?

W zadaniu 3 program znajdował **jeden pierwiastek** wielomianu.

W zadaniu 4 trzeba znaleźć **wszystkie pierwiastki**, czyli nie kończyć programu po jednym, tylko szukać kolejnych.

Robimy to tak:

1. Szukamy pierwszego pierwiastka metodą Laguerre'a.
2. Po znalezieniu pierwiastka z_1 obniżamy stopień wielomianu, czyli robimy deflację:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

3. Potem szukamy kolejnego pierwiastka już w nowym, mniejszym wielomianie.
4. Powtarzamy ten proces.
5. Gdy zostanie wielomian stopnia 2, rozwiązujemy go wzorem dokładnym.

Deflacja wielomianu

Co to jest deflacja?

Założmy, że znaleźliśmy pierwiastek:

$$z_0$$

Jeżeli z_0 jest pierwiastkiem wielomianu $P_n(z)$, to znaczy, że wielomian można zapisać jako:

$$P_n(z) = (z - z_0)P_{n-1}(z)$$

Czyli wielomian $P_n(z)$ dzielimy przez czynnik:

$$z - z_0$$

i otrzymujemy nowy wielomian:

$$P_{n-1}(z)$$

który ma stopień mniejszy o 1.

Dzięki temu po znalezieniu jednego pierwiastka możemy szukać kolejnego pierwiastka w wielomianie niższego stopnia.

Wyjaśnienie krok po kroku

1. Metoda Laguerre'a znajduje jeden pierwiastek

W zadaniu 3 mieliśmy funkcję:

```
metoda_laguerre_jeden_pierwiastek(...)
```

PYTHON

Ona znajduje **jeden** pierwiastek wielomianu.

Dla przykładu ze slajdu:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

Metoda startuje np. od:

$$z_0 = 0$$

znajduje pierwszy pierwiastek:

$$z_1 \approx -1$$

Skoro:

$$z_1 = -1$$

to czynnik, przez który dzielimy, ma postać:

$$z - z_1 = z - (-1) = z + 1$$

Po podzieleniu wielomianu przez $z + 1$ dostajemy:

$$P(z) = (z + 1)(z^3 + 4z^2 + 9z + 10)$$

Czyli po deflacji zostaje nam nowy wielomian:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

2. Powtarzamy szukanie pierwiastków

Teraz metodę Laguerre'a stosujemy do mniejszego wielomianu:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

Metoda może znaleźć pierwiastek zespolony, np.:

$$z_2 = -1 + 2i$$

Dla wielomianu o rzeczywistych współczynnikach drugi pierwiastek z tej pary to:

$$z_3 = -1 - 2i$$

W programie nie trzeba tego osobno dopisywać, bo metoda Laguerre'a działa na liczbach zespolonych, a deflacja pozwala dalej zmniejszać stopień wielomianu.

3. Gdy zostaje stopień 2, używamy wzoru dokładnego

Na slajdach było, że proces powtarzamy aż do wielomianu stopnia 2.

Dla wielomianu stopnia 2:

$$az^2 + bz + c = 0$$

korzystamy ze wzoru:

$$z = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

W kodzie robi to funkcja:

```
pierwiastki_stopnia_drugiego(...)
```

PYTHON

Używamy `cmath.sqrt`, ponieważ pierwiastki mogą być zespolone.

4. Strategia całego zadania

Zgodnie ze slajdami postępujemy tak:

1. Bierzemy wielomian $P_n(z)$.
2. Metodą Laguerre'a znajdujemy jeden pierwiastek.
3. Możemy go jeszcze „wygładzić”, czyli poprawić, używając go jako punktu startowego dla pierwotnego wielomianu.
4. Wykonujemy deflację:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

5. Powtarzamy obliczenia dla nowego wielomianu $P_{n-1}(z)$.

6. Kontynuujemy aż zostanie wielomian stopnia 2.

7. Dla wielomianu stopnia 2 używamy wzoru z deltą.

5. Wynik dla przykładu ze slajdu

Dla:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

dokładne pierwiastki to:

$$z_1 = -1$$

$$z_2 = -2$$

$$z_3 = -1 + 2i$$

$$z_4 = -1 - 2i$$

Program powinien wypisać wartości bardzo bliskie tym wynikom.

Kolejność może być inna, np.:

$$-1, \quad -1 + 2i, \quad -1 - 2i, \quad -2$$

i to jest normalne.

```
import cmath # potrzebne do obsługi liczb zespolonych i pierwiastka zespolonego

# ----- ZADANIE 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja oblicza wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok Hornera

    return p # zwracamy wartość wielomianu

# ----- ZADANIE 2 -----

def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja liczy P(z), P'(z), P''(z)
    p = wspolczynniki[0] # p przechowuje wartość wielomianu
    dp = 0 # dp przechowuje wartość pierwszej pochodnej
    ddp = 0 # ddp przechowuje wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
        dp = dp * z + p # aktualizujemy pierwszą pochodną
        p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)

# ----- ZADANIE 3 -----

def metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0, epsilon=1e-6, max_iteracji=100):
    z = complex(z0) # zamieniamy punkt startowy na liczbę zespoloną
    n = len(wspolczynniki) - 1 # stopień wielomianu

    for k in range(max_iteracji): # wykonujemy kolejne iteracje metody Laguerre'a
        P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z) # liczymy P(z), P'(z), P''(z)

        if abs(P) < epsilon: # jeżeli P(z) jest bardzo bliskie 0
            return z # zwracamy aktualne z jako pierwiastek

        G = P_prim / P # liczymy G = P'(z) / P(z)
        H = G**2 - P_2prim / P # liczymy H = G^2 - P''(z) / P(z)
```

```

    pierwiastek = cmath.sqrt((n - 1) * (n * H - G**2)) # liczymy wyrażenie
    pod pierwiastkiem

    mianownik_plus = G + pierwiastek # pierwszy możliwy mianownik
    mianownik_minus = G - pierwiastek # drugi możliwy mianownik

    if abs(mianownik_plus) > abs(mianownik_minus): # wybieramy mianownik o
    większym module
        mianownik = mianownik_plus # wybieramy wersję z plusem
    else:
        mianownik = mianownik_minus # wybieramy wersję z minusem

    if abs(mianownik) == 0: # zabezpieczenie przed dzieleniem przez zero
        z = z + complex(epsilon, epsilon) # lekko przesuwamy punkt startowy
        continue # przechodzimy do następnej iteracji

    a = n / mianownik # liczymy poprawkę a

    z_nowe = z - a # liczymy nowe przybliżenie pierwiastka

    if abs(a) < epsilon: # jeżeli poprawka jest bardzo mała
        return z_nowe # kończymy iteracje

    z = z_nowe # aktualizujemy z

    return z # zwracamy ostatnie przybliżenie, jeśli pętla się zakończyła

# ----- ZADANIE 4 -----

def deflacja(wspolczynniki, pierwiastek): # funkcja obniża stopień wielomianu po
    znalezieniu pierwiastka
    nowe_wspolczynniki = [complex(wspolczynniki[0])] # pierwszy współczynnik
    przepisujemy

    for i in range(1, len(wspolczynniki) - 1): # przechodzimy po współczynnikach
    oprócz ostatniego
        nowy = wspolczynniki[i] + pierwiastek * nowe_wspolczynniki[-1] #
    obliczamy kolejny współczynnik po deflacji
        nowe_wspolczynniki.append(nowy) # dodajemy go do listy nowych
    współczynników

    reszta = wspolczynniki[-1] + pierwiastek * nowe_wspolczynniki[-1] # obliczamy
    resztę z dzielenia

    return nowe_wspolczynniki, reszta # zwracamy nowy wielomian i resztę

def pierwiastki_stopnia_drugiego(wspolczynniki): # funkcja rozwiązuje wielomian
    stopnia 2
    a = wspolczynniki[0] # współczynnik przy z^2
    b = wspolczynniki[1] # współczynnik przy z
    c = wspolczynniki[2] # wyraz wolny

```

```

delta = b**2 - 4 * a * c # liczymy deltę

z1 = (-b + cmath.sqrt(delta)) / (2 * a) # pierwszy pierwiastek
z2 = (-b - cmath.sqrt(delta)) / (2 * a) # drugi pierwiastek

return z1, z2 # zwracamy dwa pierwiastki

def metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0, epsilon=1e-6):
    pierwotny_wielomian = wspolczynniki.copy() # zapamiętujemy oryginalny
    wielomian do wygładzania
    aktualny_wielomian = wspolczynniki.copy() # to będzie wielomian, którego
    stopień będziemy zmniejszać
    pierwiastki = [] # lista na znalezione pierwiastki

    while len(aktualny_wielomian) > 3: # dopóki stopień wielomianu jest większy
    niż 2
        pierwiastek = metoda_laguerre_jeden_pierwiastek(aktualny_wielomian, z0,
        epsilon) # szukamy pierwiastka

        pierwiastek = metoda_laguerre_jeden_pierwiastek(pierwotny_wielomian,
        pierwiastek, epsilon) # wygładzamy pierwiastek na pierwotnym wielomianie

        pierwiastki.append(pierwiastek) # zapisujemy znaleziony pierwiastek

        aktualny_wielomian, reszta = deflacja(aktualny_wielomian, pierwiastek) #
        wykonujemy deflację

    z1, z2 = pierwiastki_stopnia_drugiego(aktualny_wielomian) # rozwiązujemy
    pozostały wielomian stopnia 2

    pierwiastki.append(z1) # dodajemy pierwszy pierwiastek z równania kwadratowego
    pierwiastki.append(z2) # dodajemy drugi pierwiastek z równania kwadratowego

    return pierwiastki # zwracamy listę wszystkich pierwiastków

# DODATKOWA FUNKCJA (NIEPOTRZEBNA)
def ladnie(z, epsilon=1e-8): # funkcja tylko poprawia wygląd wypisywania wyników
    if abs(z.imag) < epsilon: # jeśli część urojona jest praktycznie zerem
        return z.real # zwracamy samą część rzeczywistą

    if abs(z.real) < epsilon: # jeśli część rzeczywista jest praktycznie zerem
        return complex(0, z.imag) # zwracamy liczbę z zerową częścią rzeczywistą

    return z # w innych przypadkach zwracamy liczbę bez zmian

# DODATKOWY TEST
print("----- ZADANIE 4 -----")

# Przykład ze slajdu:
#  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ 

```

```
wspolczynniki = $1, 5, 13, 19, 10$ # współczynniki wielomianu od najwyższej
potęgi do wyrazu wolnego

pierwiastki = metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0) #
szukamy wszystkich pierwiastków

print("Wielomian: P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10") # wypisujemy wielomian

print("\nZnalezione pierwiastki:") # nagłówek
for i in range(len(pierwiastki)): # przechodzimy po znalezionych pierwiastkach
    print(f"z{i + 1} =", ładnie(pierwiastki[i])) # wypisujemy pierwiastek
```

Najważniejsze wnioski

W zadaniu 4 metoda działa tak:

Laguerre → pierwiastek → deflacja → kolejny pierwiastek

Czyli po każdym znalezionym pierwiastku obniżamy stopień wielomianu i szukamy dalej.

Metoda działa również dla pierwiastków zespolonych, ponieważ w programie używamy typu `complex` oraz biblioteki:

`cmath`

PYTHON

Dzięki temu program może znaleźć pierwiastki typu:

$$-1 + 2i$$

oraz:

$$-1 - 2i$$

Zadanie 5

Polecenie: Wyniki powyższych programów przetestuj dla następujących wielomianów:

a)

$$w(x) = x^3 - 6x^2 + 11x - 6$$

b)

$$w(x) = x^3 - 6x^2 + 11x - 1$$

c)

Przykład ze strony 27 wykładu:

$$Q(x) = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x - 280$$

albo w postaci iloczynowej:

$$Q(x) = 17^3 \cdot 19 \cdot 20 \cdot 21 \left(x + \frac{20}{21}\right) \left(x - \frac{16}{17}\right)^3 \left(x - \frac{18}{19}\right) \left(x - \frac{19}{20}\right)$$

d)

Przykład ze strony 27 wykładu (+1):

$$Q(x) + 1 = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x -$$

```
# zad 5 -----
```

```
def testuj_wielomian(nazwa, wspolczynniki):
    print("\n" + "=" * 60)
    print(nazwa)
    print("Współczynniki:", wspolczynniki)

    z = complex(2, 0)

    print("\nZadanie 1 - wartość wielomianu w punkcie z = 2:")
    print("P(2) =", ladnie(schemat_Hornera(wspolczynniki, z)))

    print("\nZadanie 2 - wartość P(2), P'(2), P''(2):")
    P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z)

    print("P(2)   =", ladnie(P))
    print("P'(2)  =", ladnie(P_prim))
    print("P''(2) =", ladnie(P_2prim))

    print("\nZadanie 3 - jeden pierwiastek metodą Laguerre'a:")
    jeden_pierwiastek = metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0=0)
    print("Jeden pierwiastek =", ladnie(jeden_pierwiastek))

    print("\nZadanie 4 - wszystkie pierwiastki metodą Laguerre'a:")
    pierwiastki = metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0)

    for i in range(len(pierwiastki)):
        print(f"z{i + 1} =", ladnie(pierwiastki[i]))

# a)  $w(x) = x^3 - 6x^2 + 11x - 6$ 
wielomian_A = [1, -6, 11, -6]

# b)  $w(x) = x^3 - 6x^2 + 11x - 1$ 
wielomian_B = [1, -6, 11, -1]

# c) przykład ze strony 27 wykładu
wielomian_C = [
    39205740,
    -147747493,
    173235338,
    2869080,
    -158495872,
    118949888,
    -28016640
]

# d) przykład ze strony 27 wykładu + 1
wielomian_D = [
    39205740,
    -147747493,
    173235338,
```



```

2869080,
-158495872,
118949888,
-28016639
]

testuj_wielomian(
    "a)  $w(x) = x^3 - 6x^2 + 11x - 6$ ",
    wielomian_A
)

testuj_wielomian(
    "b)  $w(x) = x^3 - 6x^2 + 11x - 1$ ",
    wielomian_B
)

testuj_wielomian(
    "c) przykład ze strony 27 wykładu",
    wielomian_C
)

testuj_wielomian(
    "d) przykład ze strony 27 wykładu + 1",
    wielomian_D
)

```

Krótką notatka do zadania 5

Testowane są wielomiany:

$$w(x) = x^3 - 6x^2 + 11x - 6$$

$$w(x) = x^3 - 6x^2 + 11x - 1$$

oraz przykład ze strony 27 wykładu:

$$Q(x) = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x - 28016640$$

Ostatni wielomian to przykład ze strony 27 wykładu z dodaną wartością 1, czyli zmieniamy tylko wyraz wolny:

$$-28016640 + 1 = -28016639$$

Dlatego w programie zapisujemy go jako:

```
wielomian_D = [  
    39205740,  
    -147747493,  
    173235338,  
    2869080,  
    -158495872,  
    118949888,  
    -28016639  
]
```

Dla każdego wielomianu program sprawdza:

1. wartość wielomianu w punkcie $z = 2$ schematem Hornera,
2. wartość $P(2)$, $P'(2)$, $P''(2)$,
3. jeden pierwiastek metodą Laguerre'a,
4. wszystkie pierwiastki metodą Laguerre'a z deflacją.

Wyniki pierwiastków mogą pojawić się w różnej kolejności. Małe części urojone typu $10^{-10}i$ przy pierwiastkach rzeczywistych można traktować jako błąd numeryczny, czyli praktycznie zero.